

Circular Lists

i) Insertion at start

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* insertAtStart(struct Node* head, int data) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = data;
12     if (head == NULL) {
13         newNode->next = newNode->prev = newNode;
14         return newNode;
15     }
16     struct Node* last = head->prev;
17     newNode->next = head;
18     newNode->prev = last;
19     last->next = head->prev = newNode;
20     return newNode;
21 }
22
23 void display(struct Node* head) {
24     if (!head) return;
25     struct Node* temp = head;
26     do {
27         printf("%d <-> ", temp->data);
28         temp = temp->next;
29     } while (temp != head);
30 }
```

```

27     printf("%d <-> ", temp->data);
28     temp = temp->next;
29 } while (temp != head);
30 printf("(head)\n");
31 }
32
33 int main() {
34     struct Node* head = NULL;
35     head = insertAtStart(head, 20);
36     head = insertAtStart(head, 10);
37     display(head);
38     return 0;
39 }

```

Output

```

10 <-> 20 <-> (head)

```

ii) Insertion at End

```
main.c    Share  Run

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* insertAtEnd(struct Node* head, int data) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = data;
12     if (head == NULL) {
13         newNode->next = newNode->prev = newNode;
14         return newNode;
15     }
16     struct Node* last = head->prev;
17     newNode->next = head;
18     newNode->prev = last;
19     last->next = head->prev = newNode;
20     return head;
21 }
22
23 int main() {
24     struct Node* head = NULL;
25     head = insertAtEnd(head, 10);
26     head = insertAtEnd(head, 20);
27     return 0;
28 }
```

iii) Insertion at Position

main.c



Share

Run

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* insertAtPos(struct Node* head, int data, int pos) {
10     if (pos == 1) {
11         struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12         newNode->data = data;
13         if (!head) { newNode->next = newNode->prev = newNode; return newNode; }
14         struct Node* last = head->prev;
15         newNode->next = head; newNode->prev = last;
16         last->next = head->prev = newNode;
17         return newNode;
18     }
19     struct Node* temp = head;
20     for (int i = 1; i < pos - 1 && temp->next != head; i++) temp = temp->next;
21     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
22     newNode->data = data;
23     newNode->next = temp->next;
24     newNode->prev = temp;
```

```

17     return newNode;
18 }
19 struct Node* temp = head;
20 for (int i = 1; i < pos - 1 && temp->next != head; i++) temp = temp
    ->next;
21 struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
22 newNode->data = data;
23 newNode->next = temp->next;
24 newNode->prev = temp;
25 temp->next->prev = newNode;
26 temp->next = newNode;
27 return head;
28 }
29
30 int main() {
31     struct Node* head = NULL;
32     head = insertAtPos(head, 10, 1);
33     head = insertAtPos(head, 30, 2);
34     head = insertAtPos(head, 20, 2);
35     return 0;
36 }

```

Output

Circular List after position insertion: 10 -> 20 -> 30 -> (head)

iv) Deletion at start

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* deleteAtStart(struct Node* head) {
10     if (head == NULL) return NULL;
11     if (head->next == head) { free(head); return NULL; }
12     struct Node* last = head->prev;
13     struct Node* temp = head;
14     head = head->next;
15     head->prev = last;
16     last->next = head;
17     free(temp);
18     return head;
19 }
20
21 int main() {
22     struct Node* head = NULL; // Assume list has items
23     head = deleteAtStart(head);
24     return 0;
25 }

```

Output

Clear

Circular List after position insertion: 10 -> 20 -> 30 -> (head)

=== Code Execution Successful ===

v) Deletion at end

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* deleteAtEnd(struct Node* head) {
10     if (head == NULL) return NULL;
11     if (head->next == head) { free(head); return NULL; }
12     struct Node* last = head->prev;
13     last->prev->next = head;
14     head->prev = last->prev;
15     free(last);
16     return head;
17 }
18
19 int main() {
20     struct Node* head = NULL;
21     head = deleteAtEnd(head);
22     return 0;
23 }

```

Output

* After deleting end, head is still: 10

=== Code Execution Successful ===

vi) Deletion at Position

main.c



Share

Run

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *next, *prev;
7  };
8
9  struct Node* deleteAtPos(struct Node* head, int pos) {
10     if (head == NULL) return NULL;
11     if (pos == 1) {
12         if (head->next == head) { free(head); return NULL; }
13         struct Node* last = head->prev;
14         struct Node* temp = head;
15         head = head->next;
16         head->prev = last; last->next = head;
17         free(temp);
18         return head;
19     }
20     struct Node* temp = head;
21     for (int i = 1; i < pos && temp->next != head; i++) temp = temp
        ->next;
22     temp->prev->next = temp->next;
23     temp->next->prev = temp->prev;
24     free(temp);
25     return head;
26 }
```



```
22     temp->prev->next = temp->next;
23     temp->next->prev = temp->prev;
24     free(temp);
25     return head;
26 }
27
28 int main() {
29     struct Node* head = NULL;
30     head = deleteAtPos(head, 2);
31     return 0;
32 }
```

Output

After deleting pos 2, list continues from 10 to: 30

=== Code Execution Successful ===